Aegon: Anonymous Execution of Governance On Noir

A project submitted for partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Information Technology

Submitted by

Shivansh Agrawal
Roll no: 12312003

Shubham
Roll no: 12312002

Under the supervision of

Dr. Sourabh Jain
Assistant Professor

Indian Institute of Information Technology, Sonepat

April 2026

Certificate

This is to certify that we have examined the project report entitled **Aegon : Anonymous Execution of Governance On Noir** has been successfully carried out by **Shivansh Agrawal** and **Shubham Shoora** under my supervision and guidance.

The work presented in this project is original and has been submitted in partial fulfillment of requirements for the award of the degree of **Bachelor of Technology (B.Tech)** in **Computer Science and Engineering**.

This work has not been presented, either in whole or in part, for the award of any other degree or diploma to this or any other institution.

Dr. Sourabh Jain
Supervisor

Department of Computer Science and Engineering
Indian Institute of Information Technology, Sonepat
Haryana, India

Self Declaration

We hereby declare that the work presented in this project titled **Aegon : Anonymous Execution of Governance On Noir** is original and carried out by **Shivansh Agrawal** and **Shubham**. We have followed all ethical guidelines in the preparation of this project, and all sources of information have been properly acknowledged.

This work has not been submitted previously, either in full or in part, for the award of any degree or diploma in any institution.

Date:

Shivansh Agrawal
Roll No : 12312003

Shubham Shoora
Roll No : 12312002

Department of Computer Science and Engineering
Indian Institute of Information Technology, Sonepat
Haryana, India

Acknowledgments

I sincerely thank **Dr. Sourabh Jain**, for invaluable guidance and mentorship throughout this project. The feedback on cryptographic systems, blockchain architecture, and decentralized governance shaped every phase of the engineering decisions made in this work.

I am also thankful to the Department of Computer Science and Engineering, Indian Institute of Information Technology, Sonepat, for providing the necessary resources and the academic environment required to carry out this project.

I also thank the open-source communities behind the Noir language (Aztec Labs), the Barretenberg proving backend, Foundry, and Ethers.js, whose well-maintained tools and documentation made the development of a functional ZK-enabled governance system achievable within the scope of a project. I acknowledge the contributors to the circomlibjs and Poseidon hash library, which are foundational to the eligibility tree construction.

Finally, we thank our families and peers for their continuous support throughout our undergraduate studies.

Shivansh Agrawal
Roll No : 12312003

Shubham
Roll No : 12312002

Abstract

Traditional online voting systems expose voter identity and preference data, creating serious privacy risks and discouraging genuine participation in governance processes. This project presents **Aegon** , a full-stack privacy-preserving decentralized autonomous organization (DAO) voting prototype that leverages zero-knowledge proofs (ZKPs) to allow eligible participants to cast verifiable anonymous votes without ever revealing their identities to the blockchain or to any observer.

The system consists of four tightly integrated components: (1) a **Noir ZK circuit** written in the Aztec Noir language that verifies Merkle membership and computes cryptographic nullifiers to prevent double voting; (2) an **off-chain eligibility builder** that constructs Poseidon-based binary Merkle trees from institutional eligibility lists and produces per-voter cryptographic credentials; (3) **on-chain Solidity smart contracts** (ZKVotingDAO and EligibilityRegistry) deployed via Foundry that validate UltraHonk proofs on-chain, manage proposal lifecycles, and enforce nullifier uniqueness; and (4) a **React + TypeScript frontend** that provides a DAO console interface for voter eligibility verification, anonymous voting via a gasless relayer, proposal creation, and on-chain governance monitoring.

The system achieves voter anonymity by ensuring that a voter's on-chain action is linked only to a cryptographic nullifier rather than their wallet address or email. The gasless relayer further decouples the voter's identity from their MetaMask account. Merkle membership proofs ensure that only pre-registered eligible participants can cast votes, and per-proposal nullifiers prevent any single voter from double-casting votes. The project demonstrates that modern ZK toolchains can be effectively integrated into end-to-end Web3 governance applications within academic constraints.

Keywords: Zero-Knowledge Proofs, Decentralized Governance, DAO Voting, Merkle Trees, Poseidon Hash, Noir Language, Nullifier, UltraHonk, Solidity, Privacy-Preserving Systems, Blockchain, Gasless Relay.

Contents

1 Introduction **1**
1.1 Introduction ... 1
1.2 Problem Statement ... 1
1.3 Project Objectives ... 2
1.4 Project Methodology ... 3
1.5 Organisation of the Report ... 3

2 Literature Review **4**
2.1 Zero-Knowledge Proofs ... 4
2.2 zk-SNARKs and zk-STARKs ... 4
2.3 Poseidon Hash Function ... 4
2.4 Merkle Trees in ZK Applications ... 4
2.5 Nullifier-Based Anonymous Voting ... 5
2.6 DAO Governance ... 5
2.7 Noir Programming Language ... 5

3 System Architecture **6**
3.1 High-Level Component Overview ... 6
3.2 Technology Stack ... 7
3.3 System Data Flow ... 7
3.4 Separation of Concerns ... 9

4 ZK Circuit Design **10**
4.1 Overview ... 10
4.2 Circuit Inputs and Outputs ... 10
4.3 Merkle Membership Verification ... 11
4.4 Nullifier Computation ... 11
4.5 Tree Depth ... 12

5 Smart Contract Design **13**
5.1 EligibilityRegistry.sol ... 13
5.2 ZKVotingDAO.sol ... 13

5.2.1 State Variables and Data Structures 14
5.2.2 Proposal Lifecycle 14
5.2.3 Vote Function 15
5.2.4 Proposer Registration via ZK Proof 16
5.3 Error Handling 16

6 Off-Chain Eligibility Builder **17**
6.1 Overview 17
6.2 Input Ingestion 17
6.3 Secret and Leaf Derivation 17
6.4 Merkle Tree Construction 18
6.5 Artifact Generation 18
6.6 Security Considerations 18

7 Frontend and Relay **19**
7.1 Frontend Architecture 19
7.2 Wallet Connection and Eligibility Verification 19
7.3 ZK Proof Generation 20
7.4 Gasless Vote Relay 20
7.5 On-Chain State Reading 21

8 Comparative Analysis **22**

9 Conclusion and Future Work **23**
9.1 Conclusion 23
9.2 Future Work 24

Bibliography **25**

List of Figures

3.1 Aegon System Architecture: Four-layer component overview showing off-chain eligibility builder, frontend UI with proof generation, gasless relay server, and on-chain smart contracts with data flow across layers. 6
3.2 End-to-End Vote Submission Flow: Chronological process showing voter email matching to credentials, local ZK proof generation, gasless relay submission, and on-chain nullifier verification. 8

4.1 Noir Circuit Logic: Private inputs (secret, path elements, indices) and public inputs (root, proposalId) flow through Merkle membership verification and nullifier computation without leaking the voter's secret. 12

5.1 Smart Contract Architecture: Relationship between EligibilityRegistry (separate authority) and ZKVotingDAO (governance engine) showing key data structures, NFT dependencies, and the separation of concerns between eligibility management and proposal voting..... 14
5.2 Proposal State Machine: The six states a proposal transitions through (Created, Pending, Active, Ended, Executed, Cancelled) governed by block timestamps and monitor administrative actions. 15

List of Tables

3.1 Aegon Technology Stack 7

5.1 Key State Variables in ZKVotingDAO..... 14
5.2 Custom Errors in ZKVotingDAO 16

6.1 Off-Chain Eligibility Artifacts..... 18

7.1 Frontend View States 19

8.1 Comparison of Aegon Against Related Systems 22

Chapter 1

Introduction

1.1 Introduction

Decentralized Autonomous Organizations (DAOs) represent a novel paradigm in organizational governance, where rules and decisions are encoded as smart contracts on a public blockchain and executed autonomously without centralized intermediaries. However, a fundamental challenge in on-chain voting is the *lack of privacy*: every vote is permanently visible on the public blockchain ledger, exposing individual voter preferences and potentially chilling participation in sensitive governance decisions.

Consider a scenario in an institutional setting such as a university DAO, where faculty or students vote on budgetary allocations, policy changes, or leadership elections. If each vote is publicly linkable to a wallet address (and by extension a person), the anonymity and fairness of the process is fundamentally compromised. Voters may fear retaliation, social pressure, or simple embarrassment, leading to strategic rather than honest voting.

Zero-knowledge proofs provide a mathematically rigorous mechanism to address this problem. A zero-knowledge proof allows a prover to convince a verifier that a statement is true without revealing any information beyond the truth of the statement itself. In the context of voting, this means a user can prove they are in the eligible set of voters and that their nullifier is correctly formed, without revealing *who* they are.

Aegon is designed to solve these challenges by providing a complete end- to-end privacy-preserving governance prototype. The system enables eligible partici- pants to cast anonymous, verifiable votes using Noir ZK circuits verified on Ethereum-compatible smart contracts. The design was inspired by privacy-preserving voting protocols such as Tornado Cash's nullifier approach and the Semaphore protocol [7].

1.2 Problem Statement

Existing on-chain DAO voting systems expose voter identities through their public wallet addresses. Even private-key pseudonymity is insufficient because blockchain analytics can often de-anonymize users by tracing transaction graphs, exchange KYC linkages, and interaction patterns.

Several specific problems motivate this project:

1. **Identity exposure**: Every `vote()` call on a standard DAO contract is permanently associated with the sender's address, making all voting decisions fully public and traceable.
2. **No eligibility privacy**: Standard allowlist-based voting systems reveal who is eligible by storing addresses on-chain, enabling external observers to track eligible voter behavior.
3. **Double-voting without nullifiers**: Simple membership proofs without nullifiers allow a voter to submit proofs multiple times for the same proposal.
4. **Gas burden on voters**: Requiring voters to pay gas fees creates friction and creates wallet-level linkage between the voter and their voting action.
5. **Eligibility centralization**: Allowing the DAO itself to control eligibility lists creates a conflict of interest, particularly in institutional governance scenarios.

These limitations demonstrate the need for a privacy-preserving, nullifier-enforced, ZK-based voting system with a dedicated eligibility authority and a gasless relay service.

1.3 Project Objectives

The primary objective of this project is to design and implement a complete ZK-based DAO voting system that provides:

1. Privacy-preserving voter authentication using Poseidon Merkle membership proofs verified by a Noir ZK circuit.
2. On-chain double-vote prevention using proposal-specific nullifiers computed as nullifier = $H(\text{secret}, \text{proposalId})$.
3. Separation of eligibility authority from the DAO proposer, through a dedicated `EligibilityRegistry` smart contract.
4. A gasless relay server so that voters need not expose their wallet address when submitting votes on-chain.
5. An end-to-end interactive React frontend that facilitates credential verification, proof generation, anonymous voting, and proposal lifecycle management.
6. An off-chain batch tool for administrators to generate eligibility Merkle trees from CSV/XLSX voter lists and produce per-voter cryptographic credential bundles.

1.4 Project Methodology

The project is organized across four engineering phases:

- **Phase 1**: ZK Circuit design in Noir — Poseidon membership proof, Merkle path verification, nullifier computation.
- **Phase 2**: Smart contract development — ZKVotingDAO.sol and EligibilityRegistry.sol with Foundry deployment scripts.
- **Phase 3**: Off-chain eligibility tree builder — CSV/XLSX ingestion, Poseidon hashing, Merkle tree construction, and per-voter credential artifact generation.
- **Phase 4**: React frontend — wallet connection, eligibility verification, ZK proof generation using @noir-lang/noir_js, gasless vote relay, and DAO dashboard.

1.5 Organisation of the Report

- **Chapter 2 – Literature Review**: Related work on zero-knowledge proofs, Merkle trees, nullifier-based anonymous voting, and DAO governance.
- **Chapter 3 – System Architecture**: Component overview, technology stack, and data flow across the four system layers.
- **Chapter 4 – ZK Circuit Design**: The Noir circuit for Merkle membership proof and nullifier computation.
- **Chapter 5 – Smart Contract Design**: ZKVotingDAO.sol, EligibilityRegistry.sol, and the on-chain verifier integration.
- **Chapter 6 – Off-Chain Eligibility Builder**: Poseidon tree construction, credential artifact generation, and admin workflow.
- **Chapter 7 – Frontend and Relay**: React DAO console, proof generation flow, and the gasless relay server.
- **Chapter 8 – Comparative Analysis**: Aegon versus related protocols.
- **Chapter 9 – Conclusion**: Contributions and future work.

Chapter 2

Literature Review

2.1 Zero-Knowledge Proofs

Goldwasser, Micali, and Rackoff [1] introduced the formal definition of zero-knowledge proofs, establishing that a prover can convince a verifier of a statement's truth without revealing any information other than the validity of the statement. This theoretical foundation underpins all modern ZK systems including SNARKs and STARKs.

2.2 zk-SNARKs and zk-STARKs

Groth [2] introduced the Groth16 zk-SNARK proving system, which produces compact constant-size proofs efficiently verifiable on-chain. Subsequent work by Gabizon et al. [3] produced the PLONK universal proving system, which removes the need for per-circuit trusted setup. The UltraHonk proving system used in this project is a successor to PLONK implemented in the Barretenberg library, providing efficient recursive-friendly proofs over the BN254 field.

2.3 Poseidon Hash Function

Graefe et al. [4] introduced the Poseidon hash function, designed specifically for efficient computation inside arithmetic circuits over prime fields. Unlike SHA-256 or Keccak, which require thousands of constraints per evaluation, Poseidon requires only a few hundred constraints, making it practically suitable for ZK proof systems. This project uses Poseidon for all leaf hashing and internal Merkle tree node computation.

2.4 Merkle Trees in ZK Applications

Binary Merkle trees [5] provide a compact commitment scheme for a set of elements such that any single element's membership can be proven with $\mathcal{O}(\log n)$ auxiliary sibling hashes. The combination of Poseidon hashing with Merkle trees is a widely adopted pattern in ZK applications including Tornado Cash [6] and Semaphore [7], and is the core eligibility mechanism in this project.

2.5 Nullifier-Based Anonymous Voting

The Tornado Cash protocol [6] pioneered the use of cryptographic nullifiers in the Ethereum context to prevent double-spending of anonymous deposits. Applied to voting, a nullifier $n = H(\text{secret}, \text{proposalId})$ binds the anonymity set (the Merkle root) to a specific voting event, so the same secret cannot produce the same nullifier twice for the same proposal. The Semaphore protocol [7] generalized this pattern as a reusable ZK identity primitive, which directly inspired the design of this project.

2.6 DAO Governance

Buterin [8] introduced the concept of Decentralized Autonomous Organizations in the Ethereum white paper. The challenges of on-chain governance, particularly voter apathy, plutocracy, and identity verification, have been extensively studied [9]. This project specifically addresses the voter privacy dimension, which has received limited engineering attention in deployed governance systems.

2.7 Noir Programming Language

The Noir language [10] is a Rust-like domain-specific language developed by Aztec Labs for writing ZK constraint systems at a high level of abstraction. Its backend-agnostic design allows the same circuit to target multiple proving systems. In this project, Noir targets the Barretenberg UltraHonk backend, enabling browser-side proof generation through WebAssembly.

Chapter 3

System Architecture

3.1 High-Level Component Overview

The Aegon system is composed of four primary components that interact across on-chain and off-chain boundaries:

1. **ZK Circuit** (zk_proof/): A Noir program that defines the mathematical constraints for a valid vote. It takes private inputs (voter secret, Merkle path) and public inputs (Merkle root, proposal ID) and produces a verifiable proof.

2. **Smart Contracts** (on-chain/contracts/): Two Solidity contracts deployed on an Ethereum-compatible chain. The EligibilityRegistry manages Merkle roots by round ID. The ZKVotingDAO manages the full proposal and voting lifecycle.

3. **Off-Chain Eligibility Builder** (off-chain/): A Node.js script that reads an institutional voter list (CSV/XLSX), constructs a Poseidon Merkle tree, and produces JSON credential files for each eligible voter.

4. **Frontend and Relay** (frontend/): A React + TypeScript application that connects to MetaMask, verifies voter eligibility, generates ZK proofs in the browser using @noir-lang/noir_js, and submits votes through a gasless relay server.

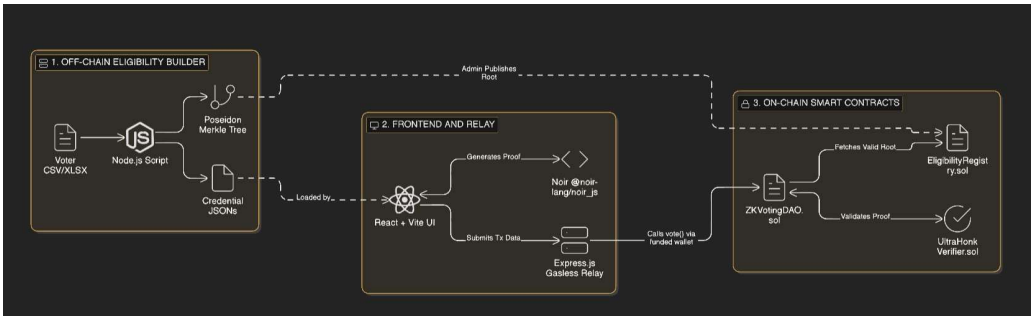


Figure 3.1: Aegon System Architecture: Four-layer component overview showing off-chain eligibility builder, frontend UI with proof generation, gasless relay server, and on-chain smart contracts with data flow across layers.

3.2 Technology Stack

Table 3.1: Aegon Technology Stack

Layer	Technology
ZK Circuit Language	Noir (Aztec Labs)
Proving Backend	Barretenberg UltraHonk (WebAssembly)
Smart Contracts	Solidity ^0.8.21
Contract Framework	Foundry (Forge, Cast, Anvil)
Frontend Framework	React 19 + Vite (TypeScript)
Blockchain Library	Ethers.js v6
Hash Function	Poseidon (BN254 field, via circomlibjs)
Relay Server	Node.js + Express.js
Eligibility Input	CSV / XLSX (via xlsx package)

3.3 System Data Flow

The complete vote submission data flow is as follows:

1. An **admin** runs the off-chain builder, ingesting a voter CSV/XLSX, building the Poseidon Merkle tree, and writing per-voter credential JSON files.
2. The admin publishes the Merkle root to `EligibilityRegistry.publishRoot(roundId, root)`.
3. A **proposer** calls `ZKVotingDAO.createProposal()`, which fetches the root from `EligibilityRegistry` and stores it in the proposal struct.
4. A **voter** opens the frontend, connects MetaMask, enters their email, and the frontend matches it to a credential in the loaded JSON artifact.
5. The voter clicks "Cast Vote". The frontend calls the Noir circuit via `@noir-lang/noir_js` in the browser to generate a UltraHonk proof.
6. The proof and public inputs are sent to the **gasless relay server**, which submits the `vote()` transaction using its own funded wallet.
7. The relay calls `ZKVotingDAO.vote()`, which: verifies the proof on-chain, checks root and proposalId match, marks the nullifier used, and records the vote tally.

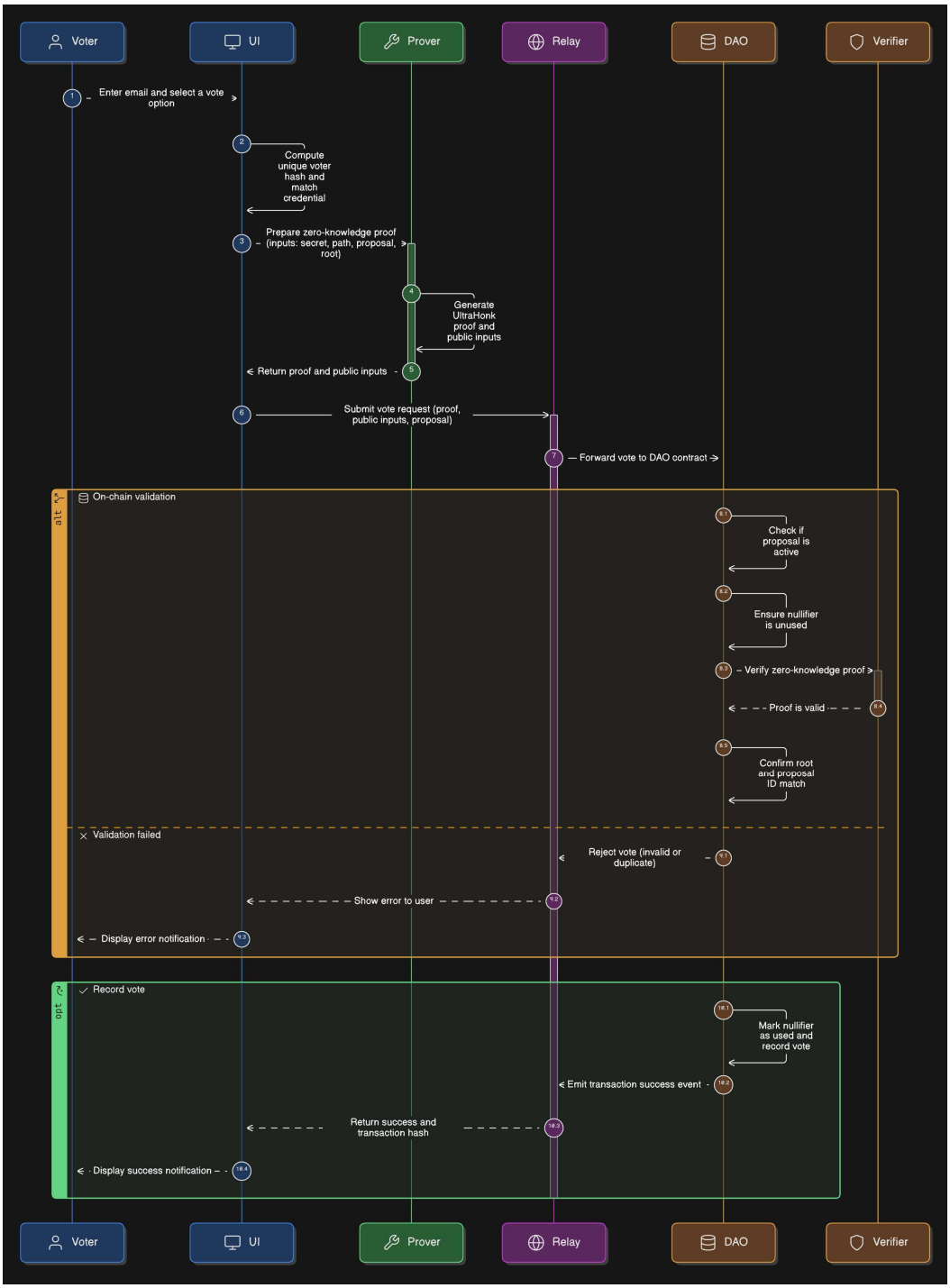


Figure 3.2: End-to-End Vote Submission Flow: Chronological process showing voter email matching to credentials, local ZK proof generation, gasless relay submission, and on-chain nullifier verification.

3.4 Separation of Concerns

A key architectural decision is the strict separation between eligibility authority and proposal authority. The `EligibilityRegistry` is owned by a separate admin who commits Merkle roots of eligible voter sets. The `ZKVotingDAO`'s monitor (the DAO admin) cannot alter eligibility roots, only reference them by round ID. This prevents the proposal owner from manipulating who can vote.

Chapter 4

ZK Circuit Design

4.1 Overview

The ZK circuit is the cryptographic core of the system. It is written in the Noir programming language and compiled to a PLONK-family arithmetic circuit representation. The compiled circuit artifact (dao.json) is loaded into the browser at runtime via WebAssembly. The circuit proves three properties simultaneously:

1. The voter knows a secret whose Poseidon hash equals a leaf in the eligibility Merkle tree.
2. The Merkle path from that leaf to the published root is valid.
3. The nullifier is computed correctly as H(secret, proposalId).

4.2 Circuit Inputs and Outputs

The circuit function signature is:

```
fn main(
    secret: Field,
    path_elements: [Field; 2],
    path_indices: [bool; 2],
    proposal_id: Field,
    root: Field,
) -> pub (Field, Field, Field)
```

Listing 4.1: Noir circuit signature (zk_proof/src/main.nr)

The inputs are divided into private and public. The secret, path_elements, and path_indices are private witness inputs known only to the prover. The proposal_id and root are public inputs that must match the on-chain proposal state.

The output is a public tuple (root, proposalId, nullifier), which becomes the publicInputs array verified by the on-chain Verifier.sol contract.

4.3 Merkle Membership Verification

The circuit computes the Merkle path traversal as follows. Given a secret, the leaf is computed as:

$$\text{leaf} = \text{Poseidon}_1(\text{secret}) \tag{4.1}$$

The circuit then traverses the Merkle path iteratively. At each level i:

$$\text{node}_{i+1} = \begin{cases} \text{Poseidon}_2(\text{node}_i, \text{path_elements}[i]) & \text{if path_indices}[i] = \text{false} \\ \text{Poseidon}_2(\text{path_elements}[i], \text{node}_i) & \text{if path_indices}[i] = \text{true} \end{cases} \tag{4.2}$$

where path_indices[i] = true means the current node is the right child (i.e., the sibling comes first). After all levels are processed, the circuit asserts:

$$\text{node}_{\text{final}} = \text{root} \tag{4.3}$$

If this assertion fails, the proof generation itself fails, so an invalid membership cannot produce a valid proof.

4.4 Nullifier Computation

The nullifier is computed as:

$$\text{nullifier} = \text{Poseidon}_2(\text{secret}, \text{proposalId}) \tag{4.4}$$

Binding the nullifier to the proposalId is critical. If the nullifier were just H(secret), using the same secret across multiple proposals would produce the same nullifier, potentially enabling cross-proposal nullifier linking that compromises anonymity. With the proposal-specific binding, each proposal produces a distinct nullifier for the same secret, preserving unlinkability across governance rounds.

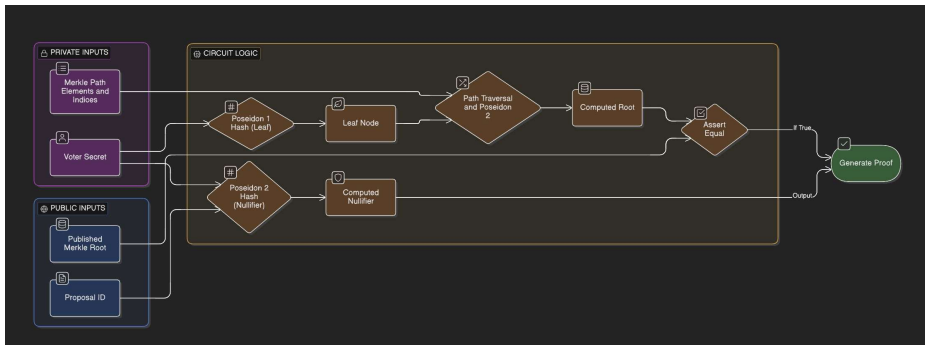


Figure 4.1: Noir Circuit Logic: Private inputs (secret, path elements, indices) and public inputs (root, proposalId) flow through Merkle membership verification and nullifier computation without leaking the voter's secret.

4.5 Tree Depth

The current circuit supports a fixed depth of 2 (`[Field; 2]` and `[bool; 2]`), supporting Merkle trees with up to 4 leaves. This is intentionally minimal for the prototype and can be extended to depth 20 (1M+ voters) by changing the array sizes and recompiling the circuit.

Chapter 5

Smart Contract Design

5.1 EligibilityRegistry.sol

The `EligibilityRegistry` contract is a simple, immutability-focused registry that stores Poseidon Merkle roots indexed by integer round IDs. Its design enforces several invariants:

- Only the contract owner can publish roots via `publishRoot(roundId, root)`.
- Once published, a root for a given `roundId` cannot be overwritten (`RootAlreadyPublished` error).
- A root of `bytes32(0)` is rejected as invalid.

This contract is deployable and ownable separately from the DAO, enforcing separation of the eligibility authority.

5.2 ZKVotingDAO.sol

The `ZKVotingDAO` contract is the core governance engine. It aggregates proposal life-cycle management, ZK proof verification, nullifier tracking, and vote tallying.

5.2.1 State Variables and Data Structures

Table 5.1: Key State Variables in ZKVotingDAO

Variable	Purpose
verifier	Immutable reference to the deployed UltraHonk verifier contract
eligibilityRegistry	Immutable reference to the EligibilityRegistry
monitor	Address with admin privileges
isProposer	Mapping of addresses allowed to create proposals
proposals	Mapping of proposalId to Proposal struct
nullifierUsed	Nested mapping: proposalId => nullifier => bool
proposerNullifierUsed	Nullifier tracking for proposer registration

The Proposal struct captures: description, yesVotes, noVotes, startTime, endTime, eligibilityRound, merkleRoot, executed, cancelled, and proposer.

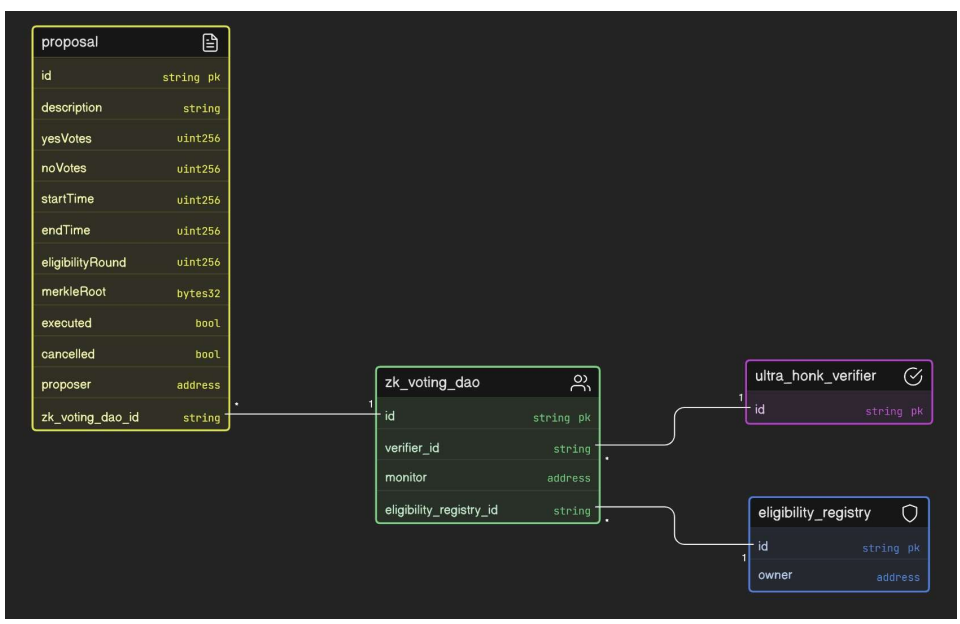


Figure 5.1: Smart Contract Architecture: Relationship between EligibilityRegistry (separate authority) and ZKVotingDAO (governance engine) showing key data structures, NFT dependencies, and the separation of concerns between eligibility management and proposal voting.

5.2.2 Proposal Lifecycle

A proposal passes through the following state machine:

1. **Created**: `createProposal()` stores the Merkle root from the registry and sets `startTime = block.timestamp + 5 minutes`.
2. **Pending**: The 5-minute delay before voting begins prevents last-minute proposer manipulation.
3. **Active**: Between `startTime` and `endTime`, eligible voters may call `vote()`.
4. **Ended**: After `endTime`, no more votes are accepted.
5. **Executed**: Any caller may invoke `executeProposal()`, which emits `ProposalExecuted(prop passed)` where `passed = yesVotes > noVotes`.
6. **Cancelled**: The monitor may cancel a proposal before `startTime`.

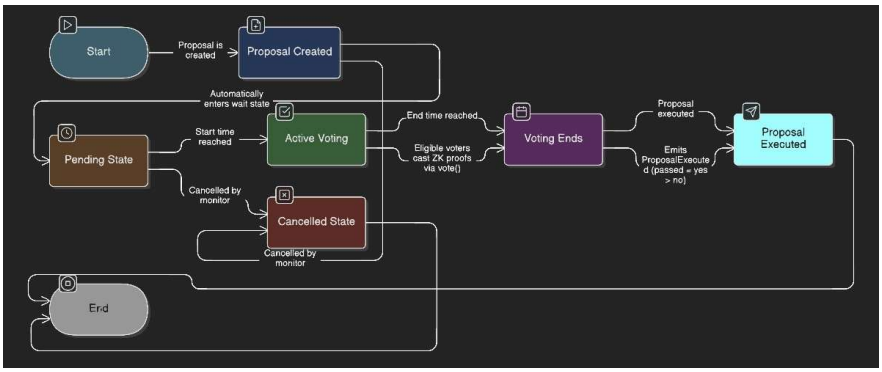


Figure 5.2: Proposal State Machine: The six states a proposal transitions through (Created, Pending, Active, Ended, Executed, Cancelled) governed by block timestamps and monitor administrative actions.

5.2.3 Vote Function

The `vote()` function is the critical security boundary of the contract. Its validation sequence is:

1. Verify proposal exists and is active (within `[startTime, endTime]`).
2. Verify `_publicInputs.length == 3`.
3. Check the nullifier `_publicInputs[2]` has not been used for this proposal.
4. Call `verifier.verify(_proof, _publicInputs)`.
5. Assert `_publicInputs[0] == proposal.merkleRoot` (root match).
6. Assert `uint256(_publicInputs[1]) == _proposalId` (proposal ID match).
7. Mark nullifier as used.
8. Increment `yesVotes` or `noVotes`.

The nullifier check before proof verification is an important gas optimization: if a double-vote is attempted, it reverts cheaply before the expensive on-chain verification call.

5.2.4 Proposer Registration via ZK Proof

The registerProposer() function allows a credential-holding user to prove membership in the proposer eligibility set (round 999) and register their wallet as a proposer. It uses the same ZK proof mechanism with a fixed PROPOSER_REGISTRATION_ID = type(uint256).max as the proposal ID in the nullifier, preventing proposer registrations from interfering with vote nullifiers.

5.3 Error Handling

The contracts use custom Solidity errors for gas-efficient reverts:

Table 5.2: Custom Errors in ZKVotingDAO

Error	Trigger Condition
InvalidProof	verifier.verify() returns false
NullifierAlreadyUsed	Nullifier has been seen for this proposal
RootMismatch	Public input root differs from stored proposal root
ProposalIdMismatch	Public input proposalId does not match
ProposalNotActive	Call outside [startTime, endTime] window
UnknownEligibilityRound	Registry has no root for the requested round
VotingAlreadyStarted	Cancel attempted after start

Chapter 6

Off-Chain Eligibility Builder

6.1 Overview

The eligibility builder (`off-chain/src/buildTrees.js`) is an administrator-facing Node.js script that automates the conversion of an institutional voter roster into cryptographic Merkle credentials. It is designed to be run once per eligibility round before any proposals referencing that round are created.

6.2 Input Ingestion

The script accepts an `--input` flag pointing to a CSV or XLSX file containing a column with an "email" header. It uses the `xlsx` library to parse the spreadsheet and extracts email addresses case-insensitively. Duplicate emails are deduplicated using a `Set`.

6.3 Secret and Leaf Derivation

For each unique email address e in the eligible set, the voter secret and leaf are derived as:

$$\mathsf{secret} = \mathrm{SHA256}(e \,\|\, \mathsf{salt}) \mod p \tag{6.1}$$

$$\mathsf{leaf} = \mathrm{Poseidon}_1(\mathsf{secret}) \tag{6.2}$$

where p is the BN254 scalar field prime:

$$p = 21888242871839275222246405745257275088548364400416034343698204186575808495617 \tag{6.3}$$

The salt is a deployment-specific secret string (e.g., `--salt department-default-salt`) provided by the admin at runtime. This prevents pre-computation attacks where an adversary tries to reverse-engineer eligible emails from their hashes.

6.4 Merkle Tree Construction

The tree is built as a balanced binary tree over the BN254 prime field using Poseidon hashing:

1. The leaf count is padded to the next power of two with zero-leaf padding: `zeroLeaf` = $\text{Poseidon}_1(0)$.
2. Internal nodes are computed bottom-up: $\text{node}_{i+1}[j] = \text{Poseidon}_2(\text{node}_i[2j], \text{node}_i[2j+1])$.
3. The root is the single node at the apex of the tree.

6.5 Artifact Generation

The script writes three JSON artifact files to the output directory:

Table 6.1: Off-Chain Eligibility Artifacts

File	Contents
`eligibility_round_N.json`	Round ID, Merkle root, depth, leaf count, timestamp
`voter_credentials_round_N.json`	Per-voter: emailHash, secret, leaf, pathElements, pathIndices
`debug_tree_round_N.json`	Full tree: root, depth, all leaves, all voter records

The `voter_credentials_round_N.json` file is distributed confidentially to eligible participants. It contains each voter's private secret and their Merkle path, which are the inputs required to generate a valid ZK proof.

6.6 Security Considerations

All BigInt values (field elements) are serialized as decimal strings to prevent JavaScript floating point precision loss. The secret derivation bounds the output to the BN254 field using modular arithmetic, ensuring compatibility with the Noir circuit's `Field` type constraints.

Chapter 7

Frontend and Relay

7.1 Frontend Architecture

The frontend is a React 19 application built with Vite and TypeScript, organized into a clean component hierarchy. The application state is managed purely with React hooks (useState, useEffect, useMemo, useCallback) without any external state library.

The application renders one of ten *view states* controlled by a top-level ViewState type:

Table 7.1: Frontend View States

View	Purpose
LANDING	Public landing page with DAO statistics
VERIFY	Email-based credential lookup and proposer ZK registration
HUB	Proposal creation and live proposal list
PROPOSAL	Detailed view of a single proposal with vote casting
ACTIVITY	Session-level activity log
EXPLORER	Filterable proposal index with search
NOTIFICATIONS	Real-time governance notification center
DOCS	In-app documentation
RESOURCES	Developer and operator resources
SUPPORT	FAQ and support links

7.2 Wallet Connection and Eligibility Verification

On wallet connection, the frontend requests MetaMask accounts via eth_requestAccounts and creates an ethers.BrowserProvider. Once connected, the application queries the ZKVotingDAO contract for the current proposal list and the user's proposer status.

Eligibility verification works entirely client-side without any network request beyond the initial credential JSON load. The voter enters their email; the frontend:

1. Normalizes the email to lowercase.

2. Computes SHA256(email) mod p to produce the emailHash field element.
3. Searches voter_credentials_1.json (round 1) and proposer_credentials_999.json (round 999) for a matching emailHash.
4. If found, stores the corresponding secret in mySecret state.

This approach means the voter's email is never transmitted anywhere. The matching is purely local and cryptographic.

7.3 ZK Proof Generation

Proof generation is performed in the browser using the @noir-lang/noir_js and @noir-lang/backe packages, which bundle the Barretenberg prover as WebAssembly.

```
const circuitInputs = {
    secret: inputs.secret.toString(),
    path_elements: inputs.pathElements.map((e) => e.toString()),
    path_indices: inputs.pathIndices,
    proposal_id: inputs.proposalId.toString(),
    root: inputs.root.toString(),
};
const { witness } = await noir.execute(circuitInputs);
const proof = await backend.generateProof(witness);
```

Listing 7.1: Proof generation in proof.ts

The witness generation (circuit evaluation) and proof generation steps are run asynchronously. The UltraHonk backend is lazily initialized on the first proof request and cached for subsequent calls.

7.4 Gasless Vote Relay

To decouple the voter's MetaMask wallet from the on-chain vote submission, the system routes vote transactions through a gasless relay server.

1. The frontend sends the proof and public inputs to POST /api/relay/vote on the relay server (http://localhost:3000).
2. The relay server decodes the request, re-encodes the public inputs as bytes32[], and calls ZKVotingDAO.vote() using its own funded private key wallet.
3. The relay returns {success: true, txHash} or an error.

The relay server applies a rate limit of 5 requests per 15 minutes per IP to prevent relay

abuse. The on-chain contract provides the ultimate security guarantee: the nullifier cannot be reused even if the relay is called multiple times, as the second call will be rejected on-chain with NullifierAlreadyUsed.

7.5 On-Chain State Reading

Proposals are fetched by polling ZKVotingDAO.proposalCount() and iterating getProposal(i) for each index. Each proposal's active state, time remaining, vote counts, and outcome are computed client-side. A 30-second interval timer updates the currentTimestamp state, triggering re-computation of proposal stages and notification generation for ending-soon and closed proposals.

Chapter 8

Comparative Analysis

Table 8.1 compares Aegon against related governance and voting systems across key dimensions.

Table 8.1: Comparison of Aegon Against Related Systems

Feature	**ZK Voting DAO**	**Compound Governor**	**Snapshot.org**	**Semaphore**
Voter privacy	Full anonymity (ZK proof)	None (public address)	None (public address)	Full anonymity
On-chain verification	Yes (Ultra-Honk)	Yes (direct)	No (off-chain)	Yes (Groth16)
Anti double-vote	Nullifier (on-chain)	Token balance	Snapshot balance	Nullifier
Eligibility control	Separate registry	Token holdings	Off-chain list	Group manager
Gasless voting	Yes (relay)	No	Yes	Depends
ZK language	Noir	N/A	N/A	Circom
Proposer ZK auth	Yes (ZK proof)	No (token)	No	No
Frontend	Full React app	Tally.xyz	Snapshot UI	Varies

Aegon is distinguished by its full voter anonymity guarantee at the on-chain protocol level, its separation of eligibility authority from proposal authority, and its ZK- based proposer registration mechanism. Unlike Semaphore, it is deployed as a self-contained DAO governance layer rather than a generic identity library.

Chapter 9

Conclusion and Future Work

9.1 Conclusion

This project presented the complete design and implementation of Aegon, a full-stack privacy-preserving decentralized governance system. The system resolves three fundamental weaknesses of existing on-chain governance: public voter identity, lack of eligibility privacy, and the burden of on-chain gas on individual voters.

The key contributions of this project are:

1. A Noir ZK circuit that simultaneously verifies Poseidon Merkle membership, validates the path from secret to the published root, and computes a proposal-bound nullifier to prevent double voting.
2. A binary Poseidon Merkle tree construction pipeline in Node.js that transforms institutional voter lists into per-voter cryptographic credential bundles (secret, index, path) over the BN254 field.
3. A two-contract Solidity architecture separating eligibility authority (`EligibilityRegistry`) from proposal governance (`ZKVotingDAO`), ensuring that the DAO proposer cannot manipulate who is eligible to vote.
4. Full on-chain ZK proof verification using the Barretenberg UltraHonk verifier, with a strict public input checking sequence (root match, proposal ID match, nullifier freshness) before any vote is recorded.
5. A gasless relay server that decouples the voter's wallet address from the on-chain vote submission, providing an additional layer of anonymity at the transaction-sender level.
6. A full-featured React + TypeScript DAO console with client-side credential matching, in-browser ZK proof generation, proposal lifecycle management, filterable explorer, and real-time notification system.
7. A ZK-based proposer registration mechanism using a dedicated eligibility round (999) with a sentinel proposal ID, allowing institutions to restrict proposal creation

to credentialed participants without exposing the proposer identity on-chain.

9.2 Future Work

- **Deeper Merkle tree support**: Extending the Noir circuit to support trees of depth 20 (up to $2^{20} \approx 1{,}000{,}000$ voters) by parameterizing the fixed array sizes, enabling real-world institutional deployment.
- **On-chain nullifier set with nullifier hash**: Replacing `bytes32` nullifier storage with a sparse Merkle tree or accumulator to enable efficient batch nullifier verification and future privacy-preserving vote counting over encrypted tallies.
- **Recursive proof aggregation**: Using Noir's recursive proof capabilities to aggregate multiple vote proofs off-chain and submit a single aggregated proof on-chain, drastically reducing gas costs at scale.
- **Quadratic voting**: Extending the vote weight model from binary (yes/no) to credit-weighted quadratic voting [11], while preserving ZK anonymity by having the circuit enforce valid credit allocation constraints.
- **Decentralized relay network**: Replacing the single-server gasless relay with a decentralized relayer network or leveraging ERC-4337 account abstraction paymasters to eliminate single points of failure and censorship in the relay layer.
- **Trusted Setup for production**: Migrating from the development UltraHonk configuration to a production-grade ceremonies-based universal structured reference string to formally eliminate trusted setup assumptions.

Bibliography

[1] S. Goldwasser, S. Micali, and C. Rackoff, "The knowledge complexity of interactive proof-systems," in *Proc. 17th Annual ACM STOC*, Providence, RI, 1985, pp. 291–304. [Online]. Available: https://doi.org/10.1145/22145.22178

[2] J. Groth, "On the size of pairing-based non-interactive arguments," in *Proc. EUROCRYPT 2016*, Vienna, Austria, May 2016, pp. 305–326. [Online]. Available: https://doi.org/10.1007/978-3-662-49896-5_11

[3] A. Gabizon, Z. J. Williamson, and O. Ciobanu, "PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge," *IACR Cryptol. ePrint Arch.*, 2019/953, 2019. [Online]. Available: https://eprint.iacr.org/2019/953

[4] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, "Poseidon: A new hash function for zero-knowledge proof systems," in *Proc. USENIX Security Symposium*, 2021, pp. 519–535. [Online]. Available: https://eprint.iacr.org/2019/458

[5] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Proc. CRYPTO 1987*, Santa Barbara, CA, Aug. 1987, pp. 369–378. [Online]. Available: https://doi.org/10.1007/3-540-48184-2_32

[6] R. Pertsev, R. Semenov, and R. Storm, "Tornado Cash Privacy Solution," 2020. [Online]. Available: https://berkeley-defi.github.io/assets/material/Tornado%20Cash%20Whitepaper.pdf

[7] Semaphore Protocol, "Semaphore: Zero-Knowledge Group Membership," 2023. [Online]. Available: https://semaphore.pse.dev

[8] V. Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," 2014. [Online]. Available: https://ethereum.org/en/whitepaper/

[9] M. Fritsch, L. Friesike, and M. Unfried, "Don't trust, verify: A guide to blockchain governance," *ACM Queue*, vol. 20, no. 3, pp. 1–36, 2022. [Online]. Available: https://doi.org/10.1145/3534777

[10] Aztec Labs, "Noir Language Documentation," 2023. [Online]. Available: https://noir-lang.org

[11] S. Lalley and E. G. Weyl, "Quadratic Voting: How Mechanism Design Can Radicalize Democracy," *AEA Papers and Proceedings*, vol. 108, pp. 33–37, May 2018. [Online]. Available: https://doi.org/10.1257/pandp.20181002

[12] R. Saulnier (ricmoo), "Ethers.js Documentation," v6, 2023. [Online]. Available: https://docs.ethers.org/v6/

[13] Paradigm, "Foundry Book – Smart Contract Development Toolchain," 2023. [Online]. Available: https://book.getfoundry.sh

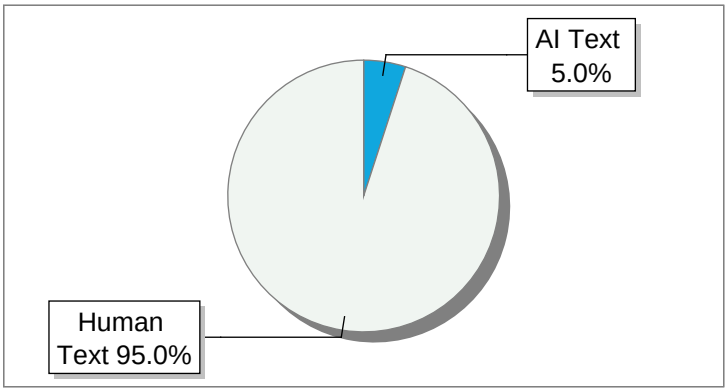
The Report is Generated by DrillBit AI Content Detection Software

Submission Information

Author Name	ShubhamAndShivansh
Title	Minor Project Report
Paper--Submission ID	5480969
Submitted By	tayyabk@iiitsonepat.ac.in
Submission Date	2026-04-13 22:43:50
Total [Pages,Sentences,Words]	[35, 255, 5059]
Document type	Others
Language	English

Result Information

AI Text: **5 %**



Disclaimer:

* The content detection system employed here is powered by artificial intelligence (AI) technology.
* Its not always accurate and only help to author identify text that might be prepared by a AI tool.
* It is designed to assist in identifying & moderating content that may violate community guidelines/legal regulations, it may not be perfect.

DrillBit

The Report is Generated by DrillBit Plagiarism Detection Software

Submission Information

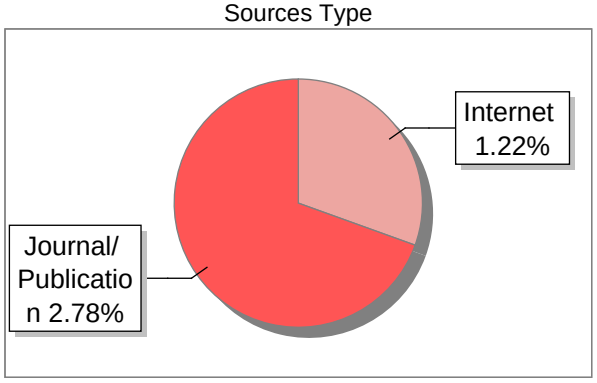
Author Name	ShubhamAndShivansh
Title	Minor Project Report
Paper/Submission ID	5480969
Submitted by	tayyabk@iiitsonepat.ac.in
Submission Date	2026-04-13 22:43:50
Total Pages, Total Words	35, 6798
Document type	Others

Result Information

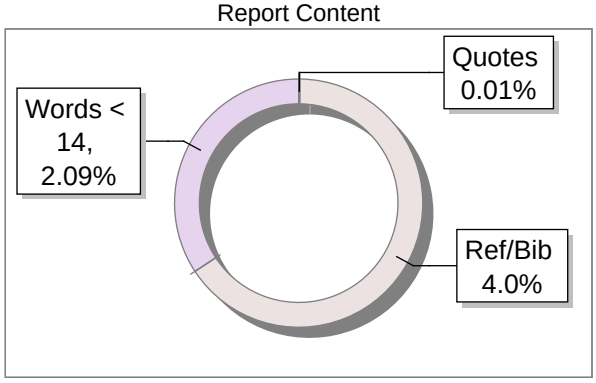
Similarity **4 %**

1 10 20 30 40 50 60 70 80 90

Sources Type



Report Content



Exclude Information

Quotes	Not Excluded
References/Bibliography	Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	**0 %**
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File